SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Scenario 1: AI-Powered Drone for Emergency Medicine Delivery (Greedy Best-First Search & A*)

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

Greedy Best-First Search (GBFS) is an informed search algorithm that selects the node with the smallest heuristic value h(n), where h(n) estimates the distance from the current node to the destination.

In the flood-affected region, the drone uses the straight-line distance to the destination as the heuristic. At every step, it moves toward the location that appears closest to the goal without considering the travel cost already incurred.

Behavior in this scenario

- The drone always chooses the path that seems nearest to the destination.
- It ignores the actual movement cost caused by weather or terrain.
- It reacts quickly because it evaluates fewer nodes.

Strengths

- Fast decision making.
- Requires less memory than many search algorithms.
- Suitable when immediate decisions are more important than perfect routes.

Weaknesses

- May choose dangerous or costly routes.
- Can get trapped if blocked paths appear.
- Does not guarantee the shortest or safest path.
- Performs poorly in uncertain environments.

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

A* Search combines:

- Actual path cost g(n)
- Estimated remaining cost h(n)

Evaluation function:

$$f(n)=g(n)+h(n)$$

where

- g(n) = Cost from start to current node
- h(n) = Estimated distance to goal

Instead of selecting only the nearest node, A* balances both the distance already travelled and the estimated remaining distance.

Behavior

Suppose two routes exist.
Route A

- Travelled cost = 8
- Remaining estimate = 3

f = 11
Route B

- Travelled cost = 4
- Remaining estimate = 9

f = 13
A* selects Route A because the total estimated cost is smaller.
Advantages

- Finds the optimal path.
- Considers blocked roads.
- Avoids unnecessarily expensive terrain.
- Much safer than Greedy Search.

iii. Analyze the impact of heuristic accuracy on both algorithms

The heuristic function estimates the remaining distance.
Highly Accurate Heuristic

- Greedy Search quickly reaches the destination.
- A* expands fewer nodes and finds the optimal path.

Poor Heuristic

- Greedy Search may move toward blocked roads.
- A* still finds the optimal solution but explores more nodes.

Comparison

Heuristic Quality	Greedy Search	A* Search
Accurate	Very Fast	Fast & Optimal
Moderate	Acceptable	Good
Poor	Poor Performance	Still Optimal but Slower

iv. Discuss how dynamic changes (blocked paths) affect search performance

Flood conditions change continuously.
Examples:

- Bridge collapses
- New flooded road
- Heavy rain
- Strong wind

Greedy Search
If a chosen path becomes blocked,

- May restart searching.
- Can waste significant time.
- Often revisits nodes.

A*
A* updates the path cost and searches for another route.
Although some additional computation is required,

- It avoids dangerous paths.
- Maintains better overall performance.

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

Recommended Algorithm: A*

Justification

Factor	Greedy	A*
Speed	Excellent	Good
Optimal Route	No	Yes
Handles Variable Cost	No	Yes
Handles Obstacles	Poor	Good
Safety	Medium	High

Emergency medicine delivery requires:

- Fast decisions
- Safe navigation
- Reliable routing

Although Greedy Search is faster, it may choose unsafe routes.
A* provides the best balance between speed and safety.
Therefore, A* is the most suitable algorithm.

Algorithm (A*) :

Start
Initialize OPEN list
Initialize CLOSED list
Insert source into OPEN
Repeat
Select node having minimum

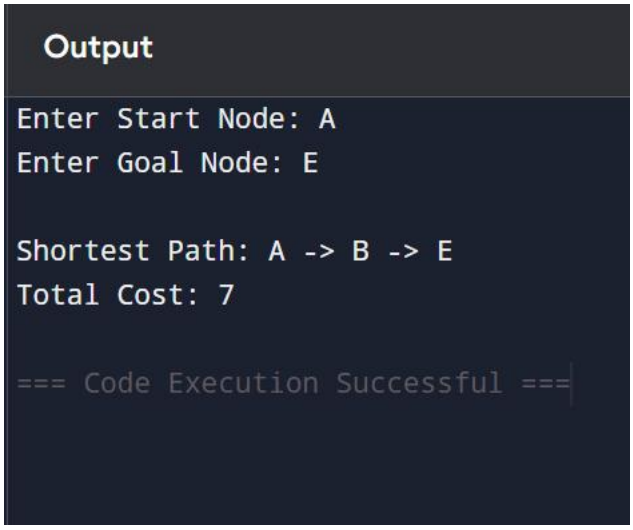
f(n)=g(n)+h(n)
If destination reached
Return path
Move node to CLOSED
For every neighbor
Calculate new g(n)
Calculate h(n)
Compute f(n)
If better path found
Update parent
Add into OPEN
Until OPEN becomes empty
Stop

CODE :

```
import heapq
graph = {
    'A': [('B', 2), ('C', 4)],
    'B': [('D', 3), ('E', 5)],
    'C': [('F', 2)],
    'D': [('G', 4)],
    'E': [('G', 2)],
    'F': [('G', 3)],
    'G': []
}
heuristic = {
    'A': 7,
    'B': 6,
    'C': 4,
    'D': 3,
    'E': 2,
    'F': 2,
    'G': 0
}
def a_star(start, goal):
    pq = []
    heapq.heappush(pq, (heuristic[start], 0, start, [start]))
    visited = {}
    while pq:
        f, g, node, path = heapq.heappop(pq)
        if node == goal:
            return path, g
```

```
        if node in visited and visited[node] <= g:
            continue
        visited[node] = g
        for neighbor, cost in graph[node]:
            new_g = g + cost
            new_f = new_g + heuristic[neighbor]
            heapq.heappush(pq, (new_f, new_g, neighbor, path + [neighbor]))
    return None
start = input("Enter Start Node: ")
goal = input("Enter Goal Node: ")
result = a_star(start, goal)
if result:
    path, cost = result
    print("\nShortest Path:", " -> ".join(path))
    print("Total Cost:", cost)
else:
    print("No Path Found")
```

OUTPUT :



Time Complexity : Worst Case: O (E log V)

Space Complexity : O(V)

Result :

The A* Search algorithm successfully determines the safest and minimum-cost path for the emergency medicine delivery drone by considering both the actual travel cost and heuristic estimate. It is more reliable than Greedy Best-First Search in dynamic environments with changing terrain and blocked paths.

Scenario 2: Smart City Traffic Signal Optimization (Hill Climbing, Simulated Annealing & Genetic Algorithm)

Scenario

A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize:

- Total waiting time
- Fuel consumption
- Traffic congestion

The system must continuously adjust signals based on traffic flow. However, the search space is extremely large, traffic patterns change dynamically, and the system may get stuck in locally optimal solutions.

Problem Formulation

The traffic signal optimization problem can be formulated as an optimization problem.

State : A state represents the signal timings at all intersections.

Example:

Intersection	Green Time (seconds)
I1	30
I2	45
I3	25
I4	40

Current State:

[30,45,25,40]

Objective Function

The objective is to minimize

Total Cost =Waiting Time +Fuel Consumption +Traffic Congestion

Lower cost means a better traffic system.

Why Traditional Search is Inefficient

Traditional search algorithms such as BFS and DFS are inefficient because:

- Hundreds of intersections create millions of possible signal combinations.
- Traffic changes continuously.
- Searching every possible state is computationally expensive.
- They cannot respond quickly to real-time traffic updates.

Hence, optimization algorithms are preferred.

i. Explain how Hill Climbing would work in this scenario and identify its limitations

Hill Climbing is a local search algorithm.
It starts with an initial traffic signal configuration and continuously moves to a neighboring configuration that improves traffic flow.

Working
Suppose the current timings are

[30,45,25,40]

Neighbor solutions

[32,45,25,40]
Cost = 260

[30,43,25,40]
Cost = 240

[30,45,28,40]
Cost = 250

Hill Climbing selects

[30,43,25,40]

because it has the minimum cost.
This process continues until no neighboring solution is better.

Advantages

- Very fast
- Easy to implement
- Uses little memory
- Suitable for continuous optimization

Limitations

- May stop at local optimum
- Cannot backtrack
- Depends heavily on the initial solution
- May miss the global optimum

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

1. Local Maximum (Local Optimum)
The algorithm reaches a solution where every neighboring solution is worse.
Example:
Current Waiting Time

200 seconds
Neighbor 1 = 210
Neighbor 2 = 220
Neighbor 3 = 215
Hill Climbing stops although another region may have
Waiting Time = 120 seconds

Real-world Interpretation
A traffic system may appear optimized in one city area while another configuration could reduce congestion much further.

2. Plateau
A plateau is a flat region where neighboring states have identical cost.
Example
Current Cost = 250
Neighbor 1 = 250
Neighbor 2 = 250
Neighbor 3 = 250
The algorithm cannot determine which direction to move.

Real-world Interpretation
Several traffic signal timings produce the same congestion level, making it difficult to improve.

3. Ridge
A ridge is a narrow path toward the optimum.

Example:

Current
↓
Slight improvement
↓
Large improvement
↓
Optimal solution

Hill Climbing changes only one variable at a time, making it difficult to move along the ridge.

Real-world Interpretation
Improving traffic requires adjusting multiple intersections simultaneously rather than one signal alone.

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

Simulated Annealing
Simulated Annealing occasionally accepts worse solutions.
This allows it to escape local optima.

Working
Current Cost : 200
Neighbor : 210
Hill Climbing rejects it.
Simulated Annealing may accept it temporarily.
Later it may discover
160
which is much better.

Advantages
- Escapes local optimum
- Handles dynamic environments
- Better global optimization

Genetic Algorithm
Genetic Algorithm is inspired by natural evolution.
It maintains a population of solutions.
Operations include
- Selection
- Crossover
- Mutation

Poor solutions are eliminated while better solutions survive.

Example:

Population

A = [30,45,25,40]
B = [28,42,30,38]
C = [35,40,20,42]

After crossover
Child : [30,42,25,38]
Mutation : [30,41,25,38]
The algorithm continuously improves signal timings over generations.

Advantages
- Explores many solutions simultaneously

- Escapes local optimum
- Works well for large search spaces
- Highly scalable

iv. Recommend the best approach and justify your choice based on scalability and adaptability

Recommended Algorithm: Genetic Algorithm

Justification

Feature	Hill Climbing	Simulated Annealing	Genetic Algorithm
Large Search Space	Poor	Good	Excellent
Escapes Local Optimum	No	Yes	Yes
Scalability	Low	Medium	High
Dynamic Traffic	Poor	Good	Excellent
Multiple Objectives	Poor	Good	Excellent

Since traffic optimization involves:
- Hundreds of intersections
- Dynamic traffic conditions
- Multiple optimization objectives
- Very large search space

Genetic Algorithm is the most suitable because it efficiently searches a large solution space, adapts to changing traffic conditions, and finds high-quality solutions without getting trapped in local optima.

Algorithm (Hill Climbing) :

Start
Generate an initial traffic signal configuration
Calculate its cost
Repeat
Generate neighboring configurations
Evaluate the cost of each neighbor
Select the neighbor with the lowest cost
If the neighbor improves the solution

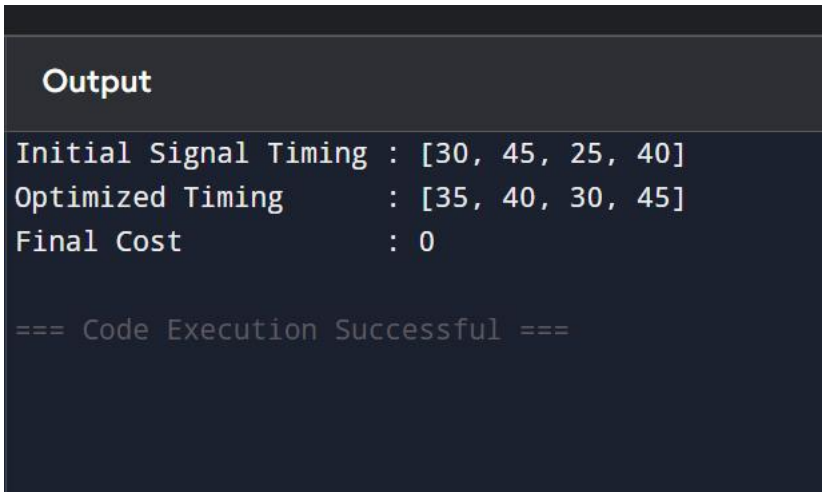
Move to that neighbor
Else
Stop
Return the best configuration
End

CODE :

```
import random
def cost(state):
    target = [35, 40, 30, 45]
    return sum(abs(state[i] - target[i]) for i in range(len(state)))
def neighbors(state):
    nbrs = []
    for i in range(len(state)):
        if state[i] + 5 <= 60:
            temp = state[:]
            temp[i] += 5
            nbrs.append(temp)
        if state[i] - 5 >= 10:
            temp = state[:]
            temp[i] -= 5
            nbrs.append(temp)
    return nbrs
def hill_climbing(initial):
    current = initial
    current_cost = cost(current)
    while True:
        nbrs = neighbors(current)
        best = current
        best_cost = current_cost
        for n in nbrs:
            c = cost(n)
            if c < best_cost:
                best = n
                best_cost = c
        if best_cost >= current_cost:
            break
        current = best
        current_cost = best_cost
    return current, current_cost
initial = [30, 45, 25, 40]
solution, final_cost = hill_climbing(initial)
```

```
print("Initial Signal Timing :", initial)
print("Optimized Timing      :", solution)
print("Final Cost            :", final_cost)
```

OUTPUT :



Time Complexity :

For **Hill Climbing**:
$O(k \times n)$
Where:

- **k** = Number of iterations
- **n** = Number of neighboring states evaluated per iteration

Space Complexity : O(n)

Result :

The Hill Climbing algorithm successfully improves traffic signal timings by iteratively selecting neighboring configurations with lower cost. However, it may become trapped in local optima. For large-scale smart city traffic optimization, **Genetic Algorithm** is the preferred approach because it offers better scalability, adaptability, and the ability to find near-optimal solutions in dynamic environments.

Scenario 3: Autonomous Mars Rover (Online Search Agent)

i. Explain why this problem requires an Online Search Agent instead of Offline Search

An Offline Search Agent requires complete knowledge of the environment before planning a path. It computes the entire route in advance and then follows it.
In contrast, the Mars rover does not have a complete map of the terrain. It discovers obstacles, rocks, and craters only as it moves. Therefore, it must make decisions based on the information currently available.
An Online Search Agent explores the environment, updates its knowledge continuously, and chooses the next action at each step.
Why Online Search is Needed

- The environment is unknown.
- Hazards are detected only during exploration.
- The rover has limited sensors.
- New information becomes available continuously.
- It must react immediately to unexpected obstacles.

Hence, an Online Search Agent is the most suitable approach.

ii. Analyze the challenges of Partial Observability and Dynamic Environments

Partial Observability : The rover cannot see the entire environment.

Examples

- Hidden craters behind hills.
- Rocks outside the sensor range.
- Unknown terrain conditions.

Challenges

- Incomplete information.
- Increased uncertainty.
- Difficult path planning.
- Higher risk of collisions.

Dynamic Environment : The environment changes while the rover is operating.
Examples

- Dust storms reduce visibility.
- Loose sand changes terrain.
- Newly detected obstacles.

Challenges

- Planned paths become invalid.
- Frequent replanning is required.
- Increased computation.
- Safety must be maintained.

iii. Describe how the rover builds and updates its internal model

The rover maintains an internal map of the explored area.
Step 1: Sense
The rover scans nearby cells using its sensors.
Example

Unknown Unknown Unknown

Unknown Rover Rock

Unknown Unknown Unknown

Step 2: Update Model

The detected rock is stored in memory.

Unknown Unknown Unknown

Unknown Rover Rock

Unknown Unknown Unknown

Step 3: Plan
The rover selects the safest unexplored path.

Step 4: Move
The rover moves one step.

Step 5: Repeat
The process continues until the destination or sample location is reached.
Thus, the internal model grows as exploration continues.

iv. Compare Online Search with Uninformed and Informed Search Strategies

Feature	**Online Search**	**Uninformed Search**	**Informed Search**
Complete Map Required	No	Yes	Yes
Uses Heuristic	Optional	No	Yes
Works in Unknown Environment	Yes	No	Limited
Real-Time Decisions	Yes	No	Limited

Feature	Online Search	Uninformed Search	Informed Search
Adaptability	High	Low	Medium
Suitable for Mars Rover	Yes	No	Partially

Explanation

- Uninformed Search (BFS, DFS) assumes the environment is known beforehand.
- Informed Search (Greedy Search, A*) uses heuristics but still benefits from a known map.
- Online Search explores and learns while moving, making it ideal for unknown environments like Mars.

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

Recommended Approach: Online Search Agent

Justification

Factor	Online Search
Unknown Environment	Excellent
Real-Time Decision Making	Excellent
Handles Uncertainty	Excellent
Learns While Exploring	Yes
Safety	High
Adaptability	High

The Mars rover cannot rely on precomputed paths because the environment is unknown. An Online Search Agent continuously senses, updates its internal model, and replans its route, ensuring safe navigation and successful mission completion.

Algorithm (Online Search Agent):

Start
Initialize an empty map
Set current position as Start
Repeat

Sense nearby environment
Update internal map
If goal is detected
Move towards goal
Else
Select the safest unexplored neighboring cell
Move to the selected cell
Repeat until goal is reached
Stop

CODE :

```
from collections import deque
UNKNOWN = -1
FREE = 0
OBSTACLE = 1
GOAL = 2
actual_map = [
    [0, 0, 1, 0, 0],
    [1, 0, 1, 0, 2],
    [0, 0, 0, 0, 1],
    [0, 1, 1, 0, 0],
    [0, 0, 0, 0, 0]
]
rows = len(actual_map)
cols = len(actual_map[0])
known_map = [[UNKNOWN for _ in range(cols)] for _ in range(rows)]
start = (0, 0)
def sense(position):
    x, y = position
    for dx, dy in [(-1,0),(1,0),(0,-1),(0,1),(0,0)]:
        nx, ny = x + dx, y + dy
        if 0 <= nx < rows and 0 <= ny < cols:
            known_map[nx][ny] = actual_map[nx][ny]
def bfs(start):
    queue = deque()
    queue.append((start, [start]))
    visited = set()
    while queue:
        (x, y), path = queue.popleft()
        if (x, y) in visited:
            continue
        visited.add((x, y))
```

```
        sense((x, y))
        if known_map[x][y] == GOAL:
            return path
        for dx, dy in [(-1,0),(1,0),(0,-1),(0,1)]:
            nx, ny = x + dx, y + dy
            if 0 <= nx < rows and 0 <= ny < cols:
                if actual_map[nx][ny] != OBSTACLE:
                    queue.append(((nx, ny), path + [(nx, ny)]))
    return None
path = bfs(start)
print("Path Followed by Rover:")
if path:
    for step in path:
        print(step)
else:
    print("Goal Not Found")
print("\nKnown Map After Exploration:")
for row in known_map:
    print(row)
```

OUTPUT :

```
Output
Path Followed by Rover:
(0, 0)
(0, 1)
(1, 1)
(2, 1)
(2, 2)
(2, 3)
(1, 3)
(1, 4)

Known Map After Exploration:
[0, 0, 1, 0, 0]
[1, 0, 1, 0, 2]
[0, 0, 0, 0, 1]
[0, 1, 1, 0, 0]
[0, 0, 0, 0, -1]

=== Code Execution Successful ===
```

Time Complexity
Using BFS for exploration: O (V + E)
Where:

- **V** = Number of cells (vertices)
- **E** = Number of possible moves (edges)

Space Complexity : O(V)

Result :

The Online Search Agent successfully explores the unknown Martian terrain, updates its internal map using sensor information, avoids obstacles, and safely reaches the goal. Compared to offline, uninformed, and informed search methods, it is the most appropriate approach because it supports real-time decision-making, adapts to newly discovered information, and ensures safe navigation in uncertain environments.

Scenario 4: University Exam Timetable Generation (Constraint Satisfaction Problem - CSP)

Problem Formulation (CSP)

- Variables: Exams (CS101, MA101, AI101, etc.)
- Domains: Available time slots (T1, T2, T3) and exam halls.
- Constraints:
    - No student has overlapping exams.
    - Hall capacity must be sufficient.
    - Faculty must be available.
    - Some subjects must be scheduled before others.
    - Special accommodation requirements must be met.

i. Variables, Domains, and Constraints

- Variables: Exam subjects.
- Domains: Available time slots and halls.
- Constraints: No exam clashes, hall availability, faculty availability, subject order, and special accommodations.

ii. Backtracking Search
Backtracking assigns a time slot to one exam at a time. If a constraint is violated, it removes the assignment and tries another slot until a valid timetable is found.

iii. Constraint Propagation (Forward Checking)

Forward Checking removes invalid time slots from unassigned exams after each assignment. This reduces conflicts and minimizes backtracking, making the search faster.

iv. Best Strategy for Scalability
Use Backtracking + Forward Checking + MRV (Minimum Remaining Values).
Reason:

- Reduces the search space.
- Detects conflicts early.
- Generates valid timetables efficiently for a large number of courses and students.

Algorithm (Backtracking Search):

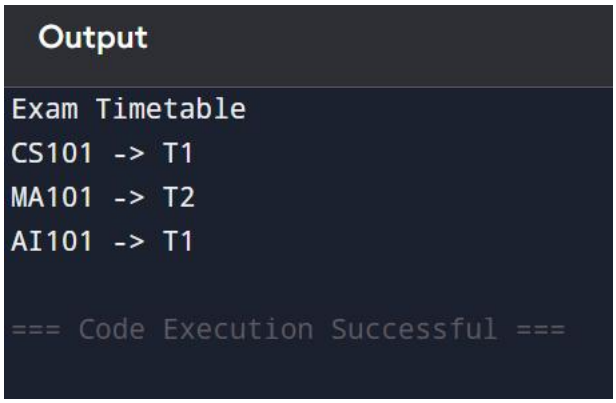
Start
Select an unassigned subject
Assign a time slot
Check all constraints
If constraints are satisfied
Assign the next subject
Else
Backtrack
Repeat until all subjects are assigned
Display timetable
Stop

CODE :

```
subjects = ["CS101", "MA101", "AI101"]
slots = ["T1", "T2", "T3"]
conflicts = {
   "CS101": ["MA101"],
   "MA101": ["CS101", "AI101"],
   "AI101": ["MA101"]
}
schedule = {}
def is_safe(subject, slot):
   for s in conflicts.get(subject, []):
      if s in schedule and schedule[s] == slot:
         return False
   return True
def solve(i):
   if i == len(subjects):
```

```
        return True
    subject = subjects[i]
    for slot in slots:
        if is_safe(subject, slot):
            schedule[subject] = slot
            if solve(i + 1):
                return True
            del schedule[subject]
    return False
if solve(0):
    print("Exam Timetable")
    for s in subjects:
        print(s, "->", schedule[s])
else:
    print("No valid timetable found.")
```

OUTPUT:



Time Complexity : O(d^n)
Space Complexity : O(n)

Result:

The CSP using Backtracking with Forward Checking efficiently generates a valid exam timetable while satisfying all constraints.

Scenario 5: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning)

i. Explain how the Minimax algorithm models this problem

Minimax is used in two-player games where one player (**AI**) tries to maximize the score, while the opponent tries to minimize it.

- **MAX node:** AI chooses the best move.

- **MIN node:** Opponent chooses the best counter-move.
- The AI selects the move with the highest guaranteed score.

ii. Analyze the computational challenges of large game trees

- A game can have thousands or millions of possible moves.
- The search tree grows exponentially with each level.
- Evaluating every possible move is slow and requires high memory.
- Real-time games have strict time limits, making full search impractical.

iii. Explain how Alpha-Beta Pruning improves efficiency

Alpha-Beta Pruning improves Minimax by ignoring branches that cannot affect the final decision.

- **Alpha (α):** Best value found for MAX.
- **Beta (β):** Best value found for MIN.
- If **α ≥ β**, the remaining branches are pruned (not explored).

Advantages:

- Reduces the number of nodes evaluated.
- Produces the same optimal result as Minimax.
- Faster decision-making for real-time games.

iv. Design an evaluation function

The evaluation function estimates how good a game state is.

Example:

Score = (10 × AI Pieces)
- (10 × Opponent Pieces)
+ (5 × Resources)
+ (8 × Territory Control)
- (6 × Damage Taken)

Factors considered:

- Number of AI units.
- Number of opponent units.
- Resources collected.
- Territory controlled.
- Health or damage status.

Higher score = Better position for the AI.

v. Recommend the best approach

Recommended Approach: Minimax with Alpha-Beta Pruning

Justification:

- Finds the optimal move.

- Reduces unnecessary computations.
- Suitable for large game trees.
- Meets real-time performance requirements.

Algorithm: Minimax with Alpha-Beta Pruning

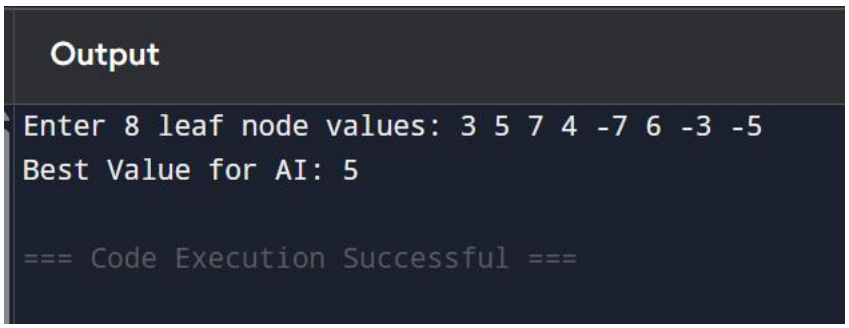
1. Start.
2. Generate the game tree from the current game state.
3. Set Alpha = −∞ and Beta = +∞.
4. If the current node is a terminal node, return its evaluation score.
5. If it is the MAX player's turn:
    - Initialize Best = −∞.
    - Evaluate each child node recursively.
    - Update Best = max(Best, Child Value).
    - Update Alpha = max(Alpha, Best).
    - If Alpha ≥ Beta, stop exploring the remaining child nodes (prune).
6. If it is the MIN player's turn:
    - Initialize Best = +∞.
    - Evaluate each child node recursively.
    - Update Best = min(Best, Child Value).
    - Update Beta = min(Beta, Best).
    - If Alpha ≥ Beta, stop exploring the remaining child nodes (prune).
7. Return the Best value to the parent node.
8. Repeat until the root node is evaluated.
9. Choose the move with the best evaluation value.
10. Stop.

CODE :

```
import math
def minimax(depth, node, maximizing, values, alpha, beta):
   if depth == 3:
      return values[node]
   if maximizing:
      best = -math.inf
      for i in range(2):
         val = minimax(depth + 1, node * 2 + i, False,
                  values, alpha, beta)
         best = max(best, val)
         alpha = max(alpha, best)
         if beta <= alpha:
            break
      return best
   else:
```

```
        best = math.inf
        for i in range(2):
            val = minimax(depth + 1, node * 2 + i, True,
                          values, alpha, beta)
            best = min(best, val)
            beta = min(beta, best)
            if beta <= alpha:
                break
        return best
values = list(map(int, input("Enter 8 leaf node values: ").split()))
result = minimax(0, 0, True, values, -math.inf, math.inf)
print("Best Value for AI:", result)
```

OUTPUT :



Time Complexity

- **Minimax:** $O(b^d)$
- **Alpha-Beta Pruning (Best Case):** $O(b^{d/2})$

where:

- **b** = Branching factor
- **d** = Depth of the game tree

Space Complexity : O(d)

Result :

The Minimax algorithm with Alpha-Beta Pruning efficiently selects the best move for the AI while reducing unnecessary node evaluations. It provides optimal decisions and is well suited for real-time strategy games with large decision trees.